

CS Study

Database - DB Connection Pool

목차

1

Connection

2

Connection Pool

3

Connection Pool 설정

목차

1

Connection

2

Connection Pool

3

Connection Pool 설정

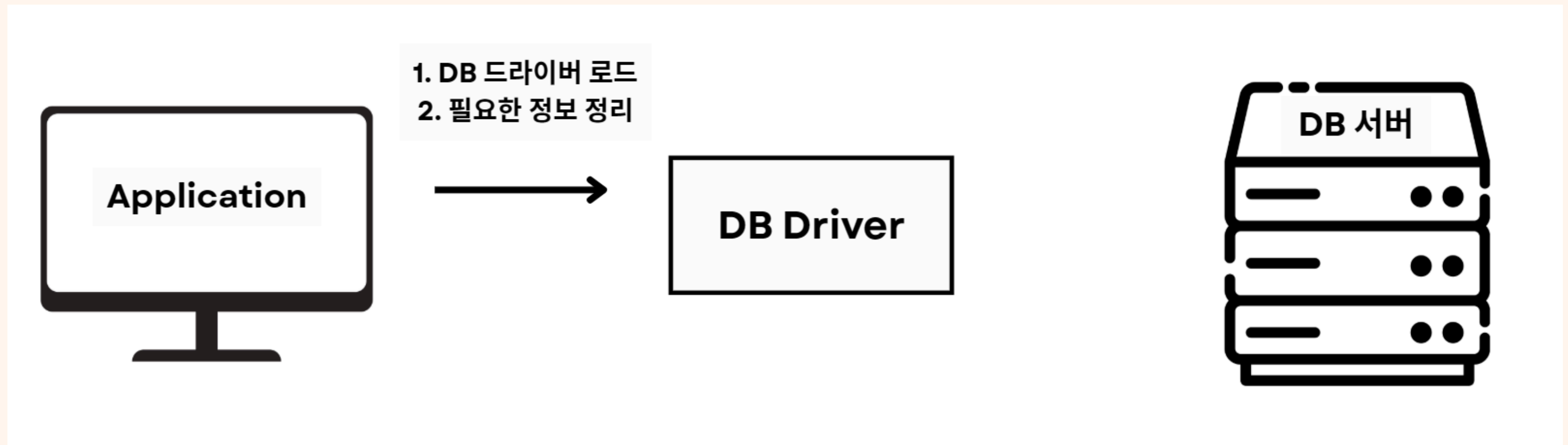
Connection이란?

Application-Database간 통신 가능하게 하는 연결 수단

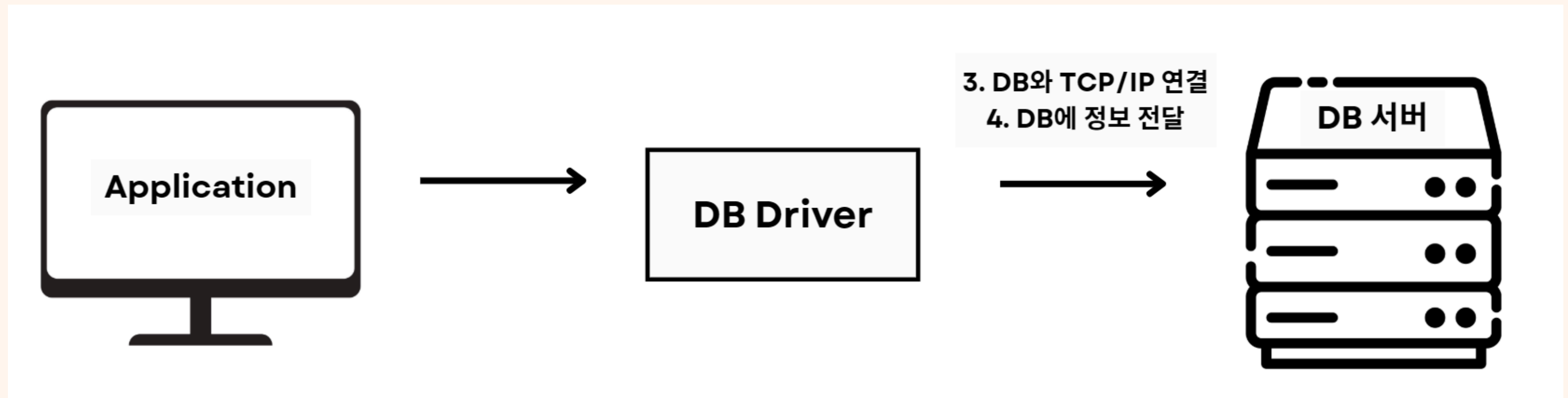
Connection 가지고 있어야 **SQL 실행, 결과 반환 가능**



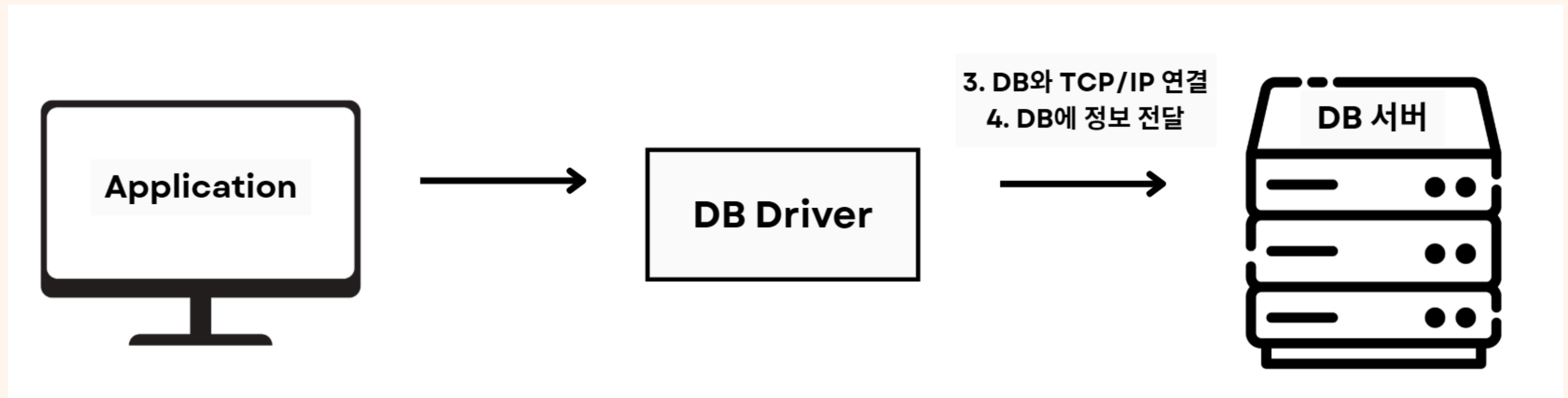
Connection 동작 과정



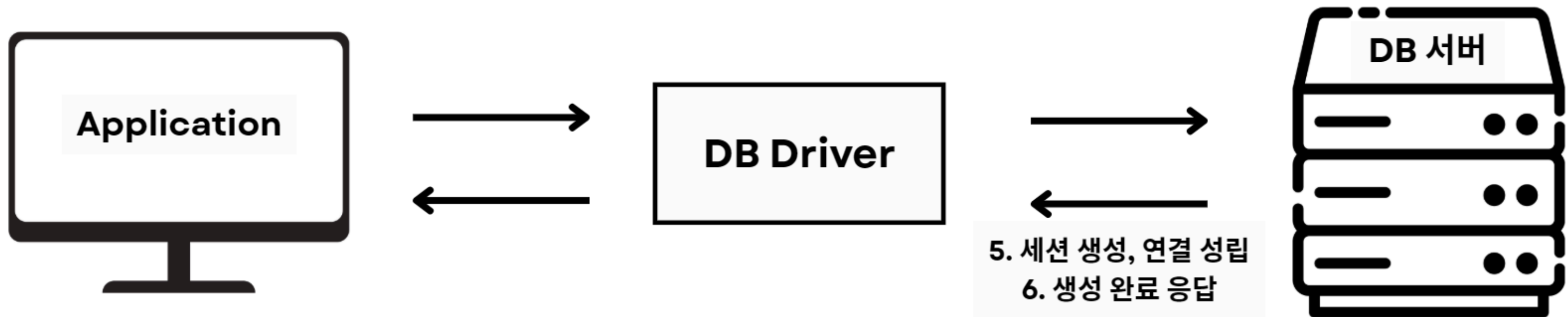
Connection 동작 과정



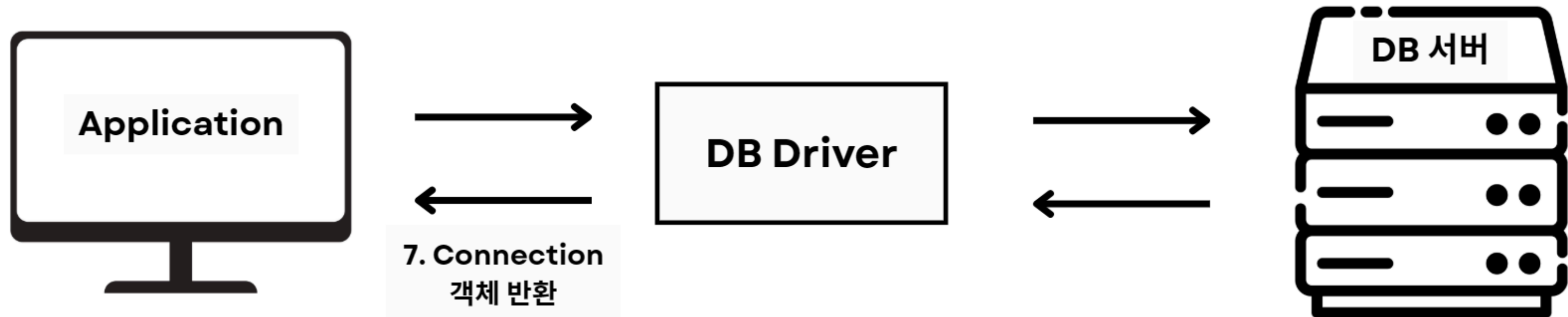
Connection 동작 과정



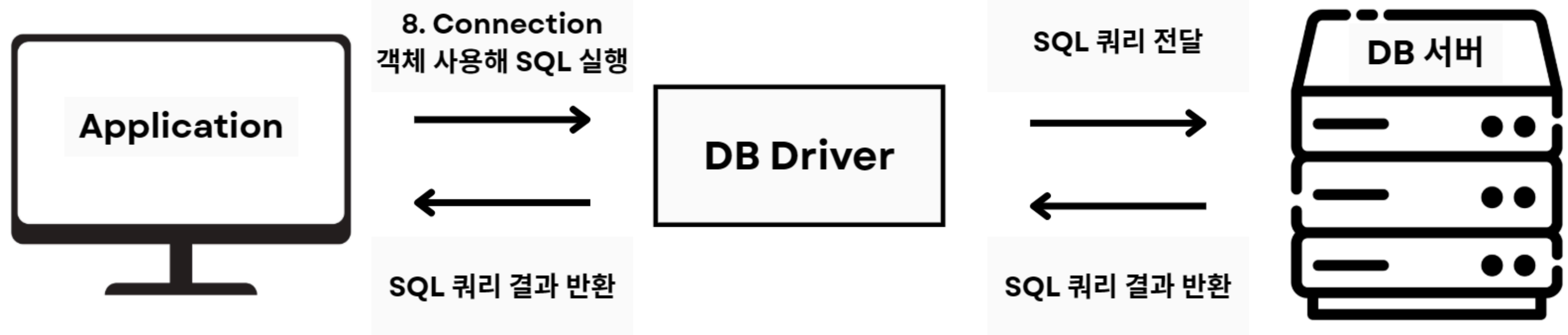
Connection 동작 과정



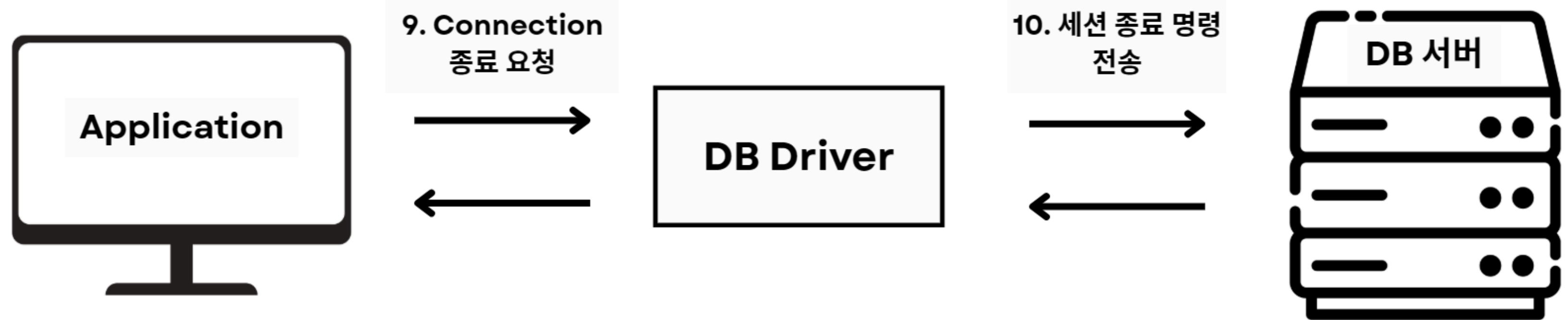
Connection 동작 과정



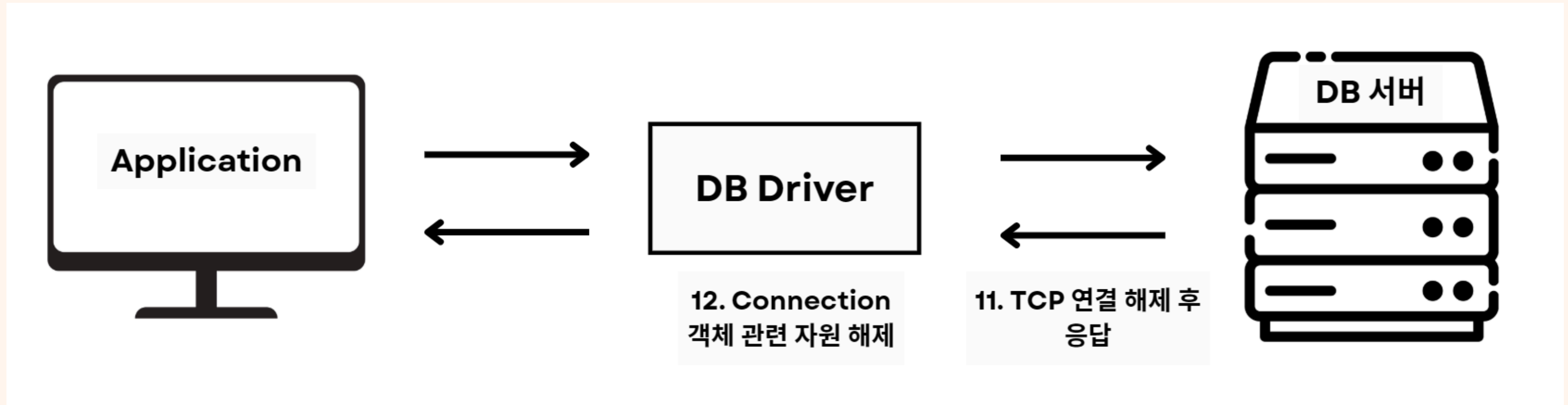
Connection 동작 과정



Connection 동작 과정



Connection 동작 과정



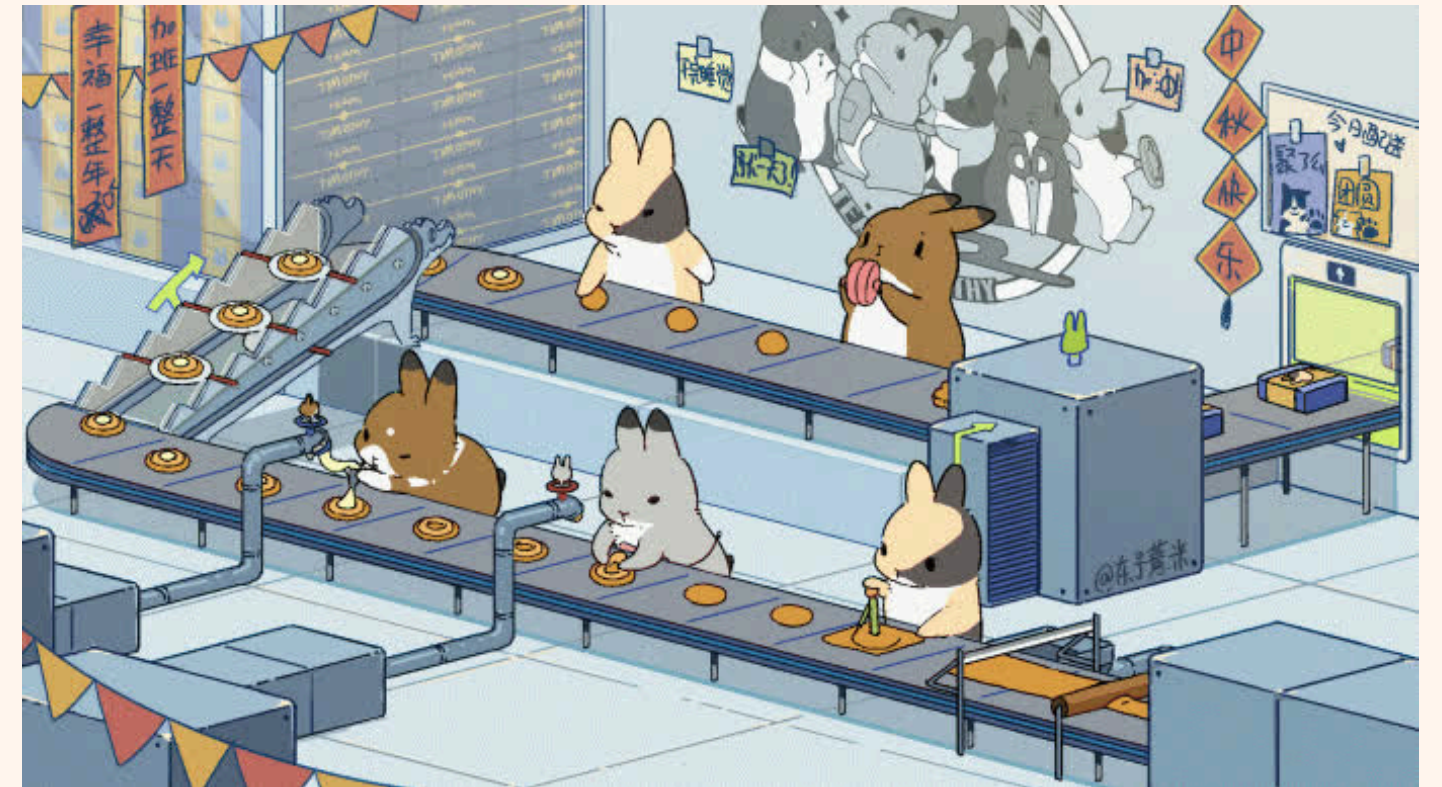
Connection만 사용 시 문제점

1. 연결 생성, 삭제 비용으로 인한 성능 저하

DB 연결은 네트워크 연결, 세션 초기화 등 필요
매 요청마다 생성 시 overhead 발생으로 성능 저하

2. DB 서버 부하 증가

DB 서버 매번 세션 생성, 삭제
연결 폭증 시 Too many connections 오류 발생



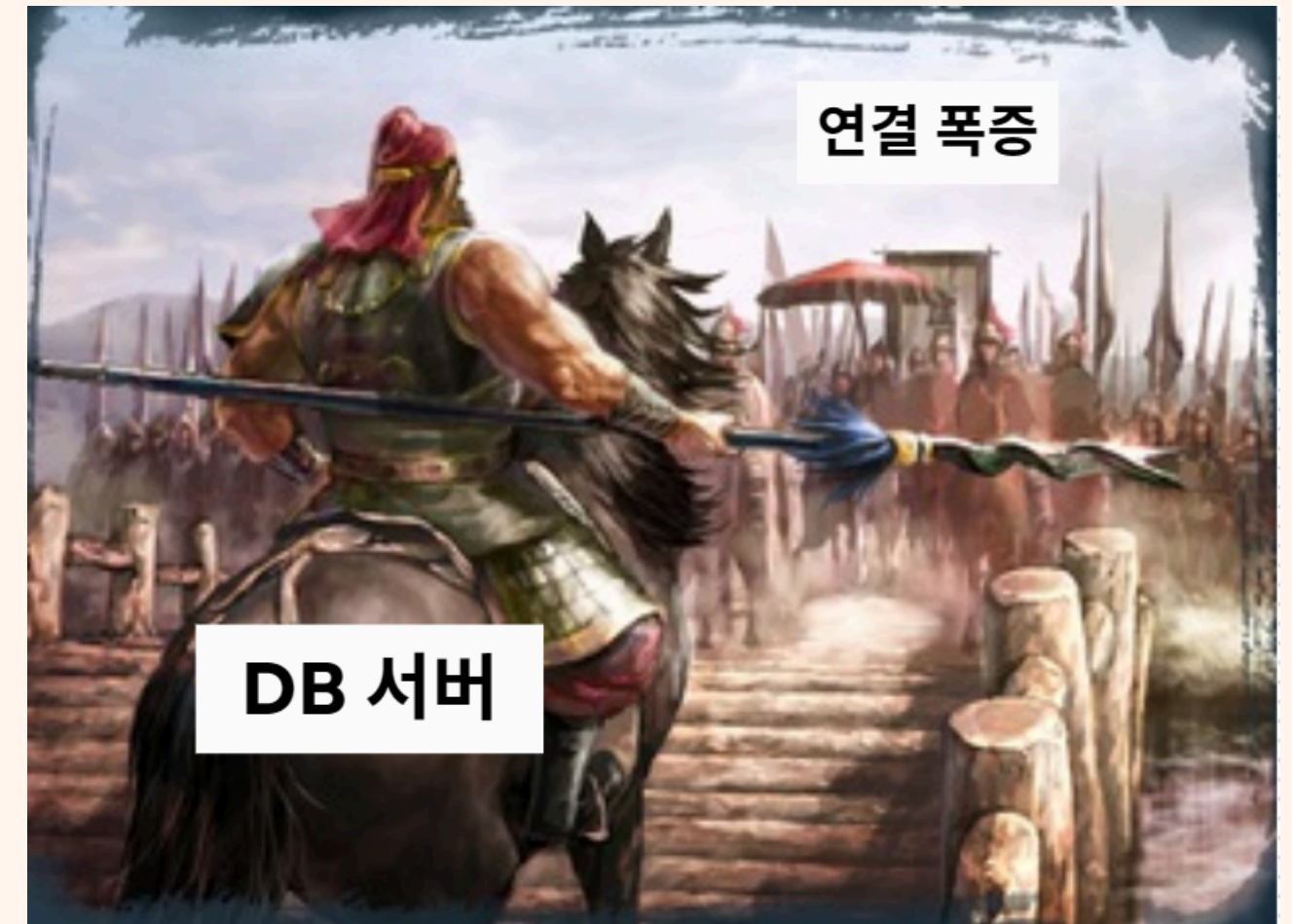
Connection만 사용 시 문제점

3. 자원 누수 및 안정성 문제

커넥션 제대로 닫지 않으면 문제 발생
커넥션 해제되지 않고 누적되어 DB 자원 낭비

4. 스파이크 트래픽 대응 불가

요청 몰리면 DB에 연결 폭탄 날려 DB 다운 가능
DB 최대 연결 수 제한에 쉽게 도달



목차

1

Connection

2

Connection Pool

3

Connection Pool 설정

Connection Pool이란

Connection Pool이란

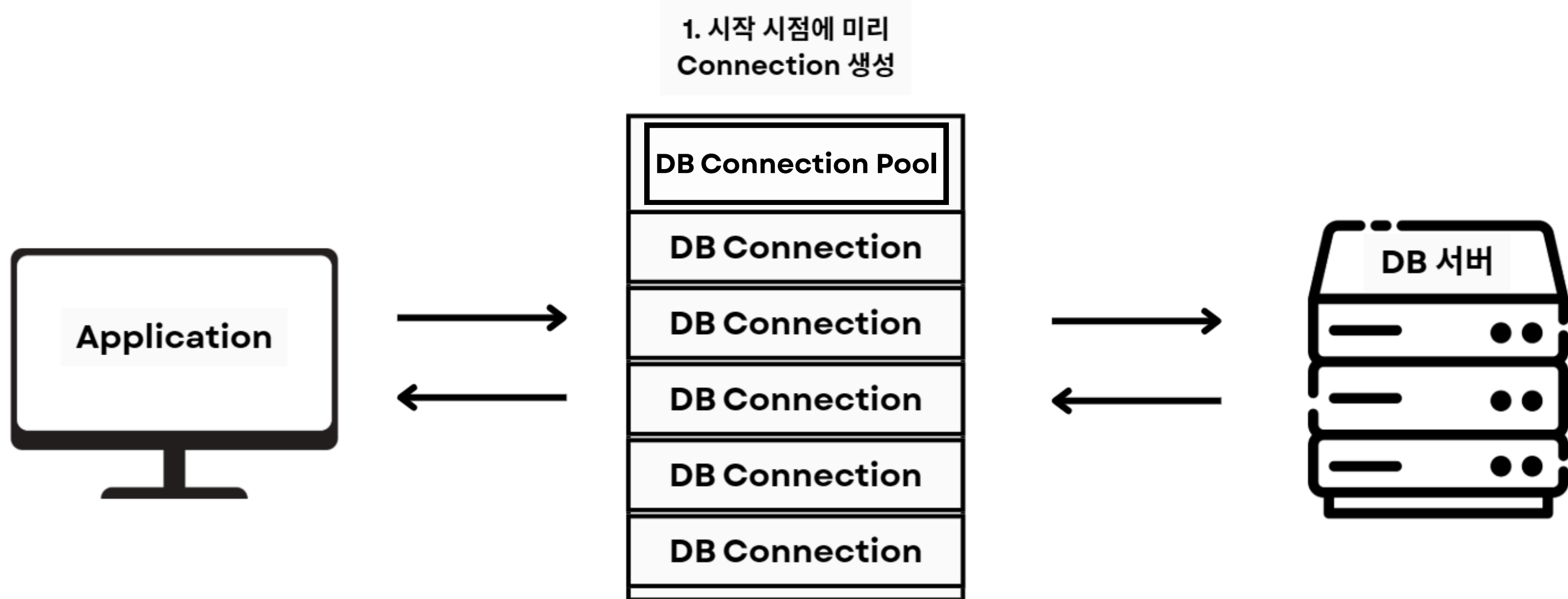
Application 실행 시 **미리 Connection 객체 생성**

생성 후 **pool에 저장**해 둬

필요 시 **Connection 할당**받아 사용한 뒤 **반환**하는 기법

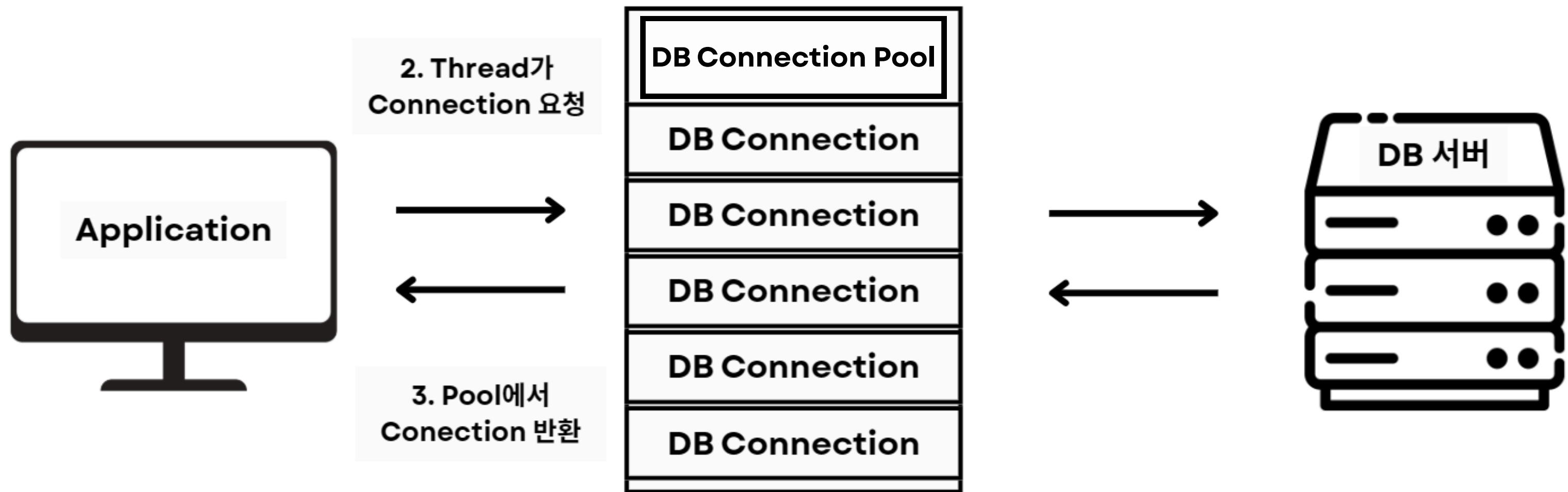


Connection Pool 동작 과정

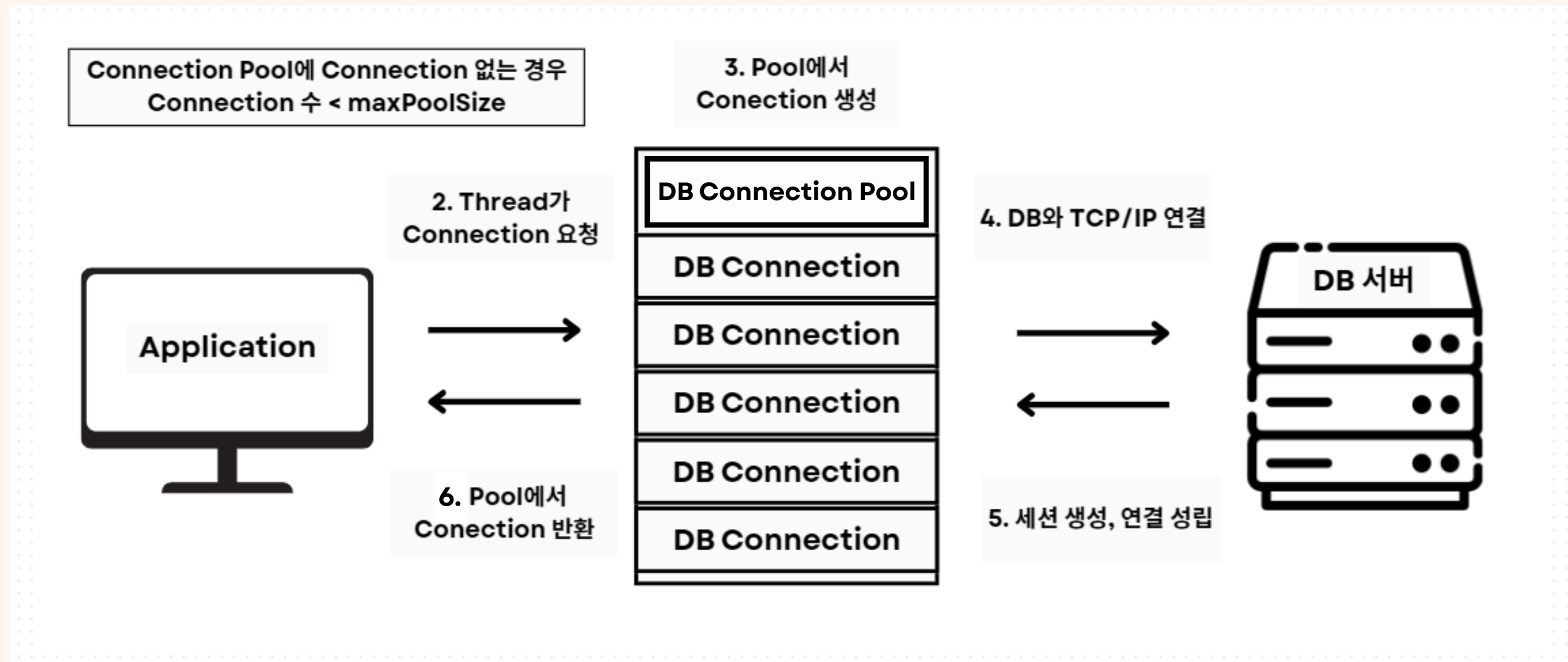


Connection Pool 동작 과정

Connection Pool에 Connection 있는 경우



Connection Pool 동작 과정

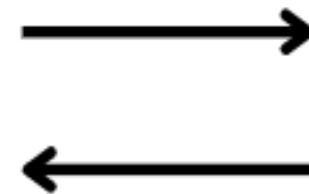


Connection Pool 동작 과정

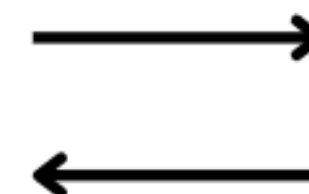
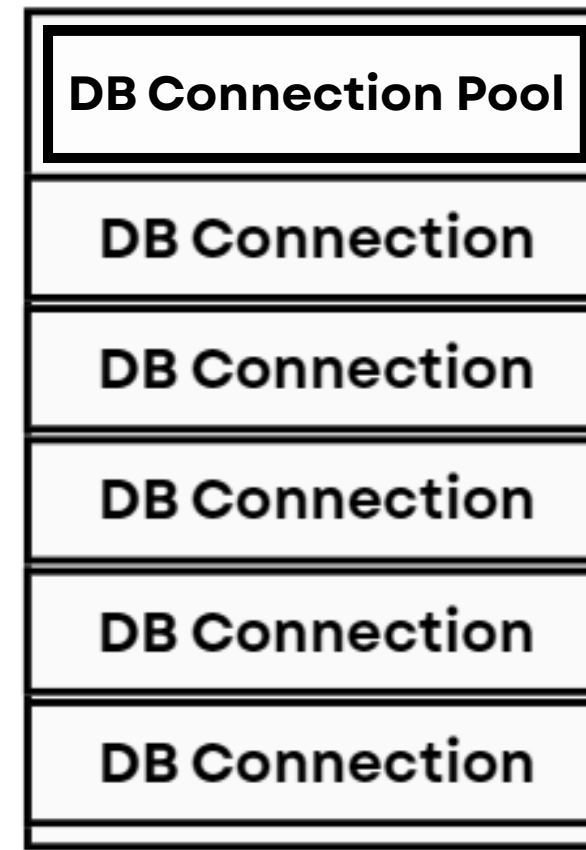
Connection Pool에 Connection 없는 경우
Connection 수 == maxPoolSize



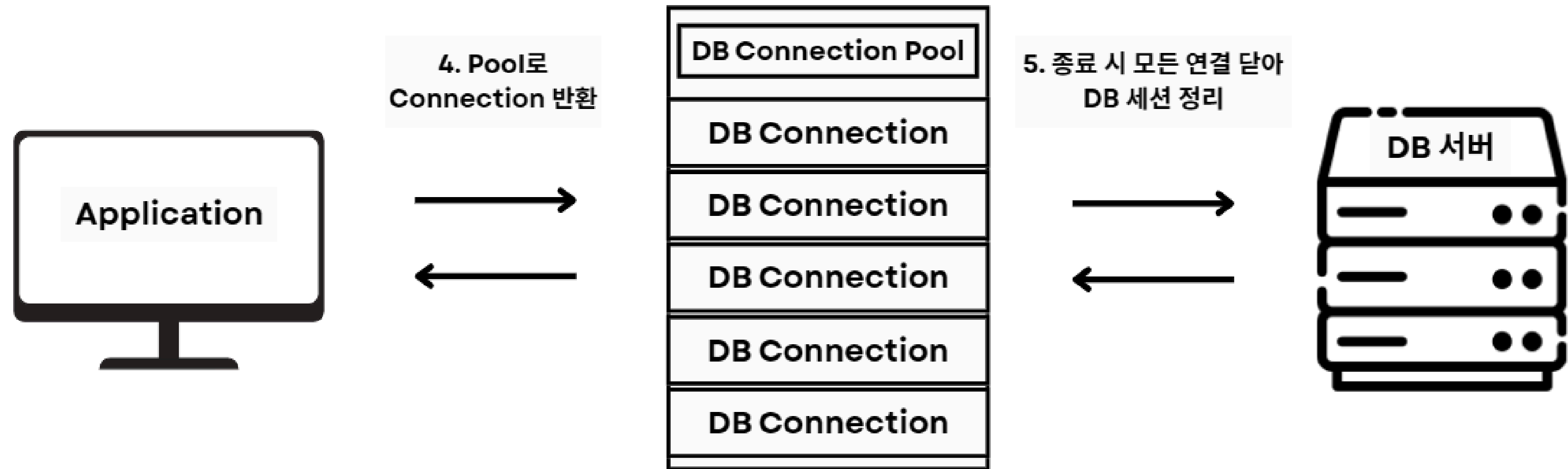
2. Thread
Connection 요청
3. Thread 대기
상태로 전환



4. Thread 시간 만료 시
예외
5. Connection 반환
되면 순차 제공



Connection Pool 동작 과정



Connection Pool 장점

1. 성능 향상

매번 Connection **생성, 삭제 비용 제거**
미리 Connection 생성해 재사용하기 때문

2. DB 서버 부하 감소

동시에 연결되는 Connection 수 제한
DB 서버에 **과도한 부하 방지**



Connection Pool 장점

3. 안정성 향상

Connection 관리 **중앙 집중화**
누수, 자원 고갈 문제 방지

4. 예측 가능한 자원 사용

Application, DB 모두 리소스 사용량
일정하게 유지 가능



Connection Pool 발생 가능 문제

1. 커넥션 수 부족 인한 대기 시간 증가

커넥션 수 < 동시 요청량

maxPoolSize를 트래픽에 맞게 조정

2. Pool 크기 너무 크게 설정

동시에 DB 연결 **과다하게** 생성 가능

DB 허용 연결 수 초과 시 **접속 거부** 가능

적절한 **pool 크기 튜닝** 필요



Connection Pool 발생 가능 문제

3. Connection 누수

마지막에 적절히 **반환하지 않은 경우**
Pool 고갈되어 **요청들 대기 상태** 됨

4. 죽은 Connection 사용

DB 서버 **Connection 끊김**
Pool은 살아있다고 인식

Connection 최대 수명 < DB 세션 timeout 설정



목차

1

Connection

2

Connection Pool

3

Connection Pool 설정

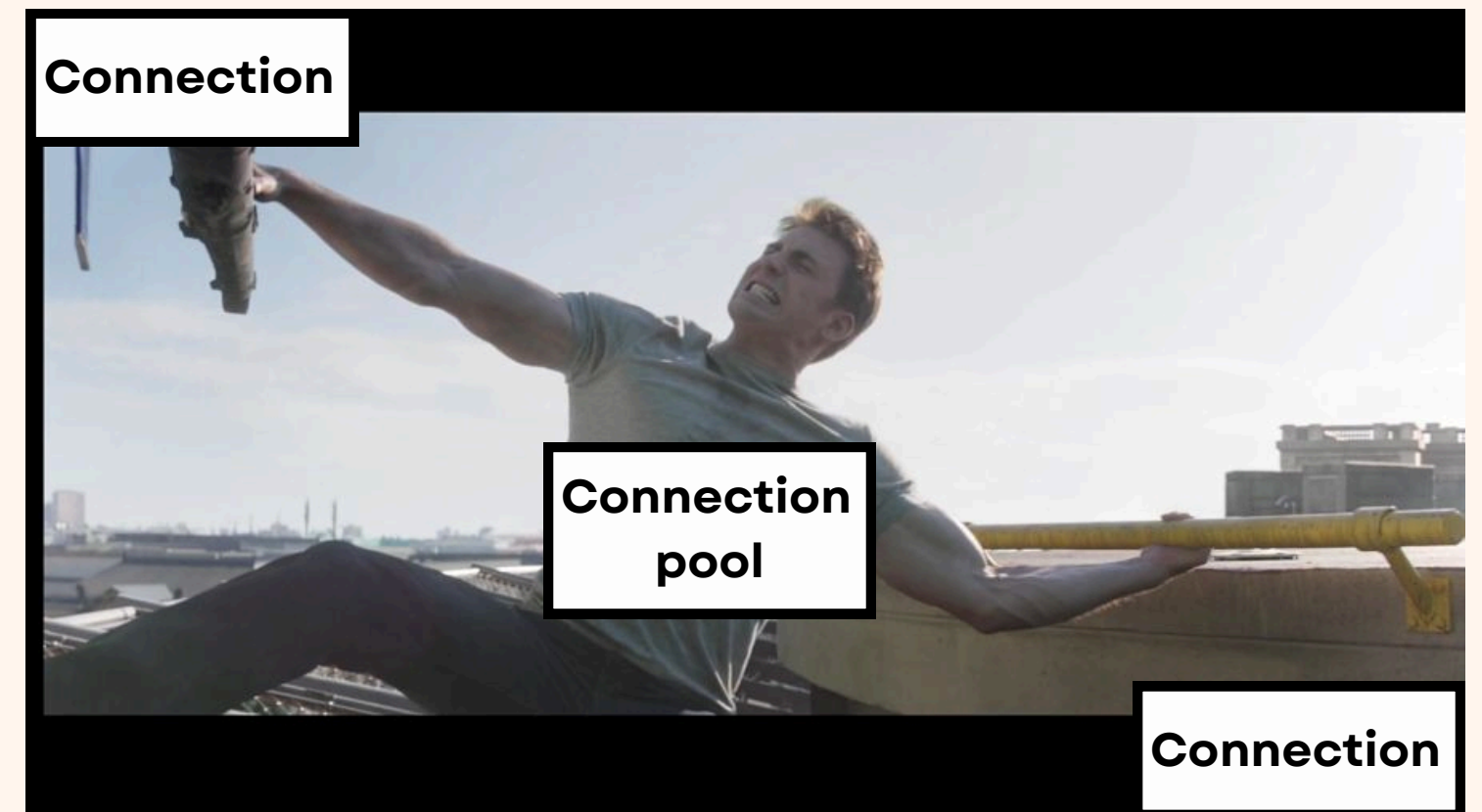
Connection Pool 설정 옵션

1. maximumPoolSize

동시에 유지 가능 최대 커넥션 수
minimumIdle 함께 보고 커넥션 추가 생성

2. minimumIdle

Pool에서 유지할 수 있는 최소 쉬고 있는 커넥션 수



Connection Pool 설정 옵션

3. idleTimeout

쉬고 있는 커넥션 유지할 최대 시간

4. connectionTimeout

커넥션 가져올 때 최대 대기 시간



Connection Pool 설정 옵션

5. maxLifetime

커넥션이 **pool**에서 존재 가능 최대 시간
DB 세션 타임아웃보다 조금 짧게 설정

6. keepaliveTime

오래된 커넥션 **연결 유지 위해 요청 보낼 시간**



Connection Pool 설정 공식

Connection Pool Size

$$= (\text{CPU 코어} * 2) + \text{effective_spindle_count}$$

effective_spindle_count = 하드디스크의 개수 또는 디스크 처리 가능 동시 I/O 요청 수

Connection Pool 설정 공식

CPU 코어 당 2배 커넥션 이유

디스크, 네트워크 IO로 인한 **blocking** 시간동안
다른 스레드 실행해 CPU 자원 효율적 사용

effective_spindle_count 더한 이유

DB의 디스크 IO 병렬성 반영하기 위함
DB가 **동시에 처리할 수 있는 것을 반영**해 늘리는 것



Quiz

1번 문제

Connection만 사용할 때 문제점으로
connection **생성, 삭제 비용으로 인한 성능 저하**가 있다.

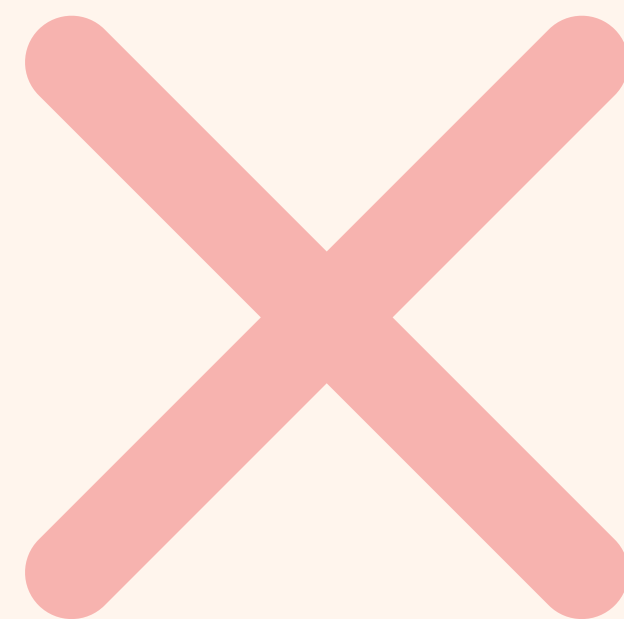


1번 문제

Connection 생성하고 삭제하는 것 **비용 굉장히 큼**

(네트워크 연결, 세션 초기화 등)

이런 일을 매번 하기에 성능 저하



2번 문제

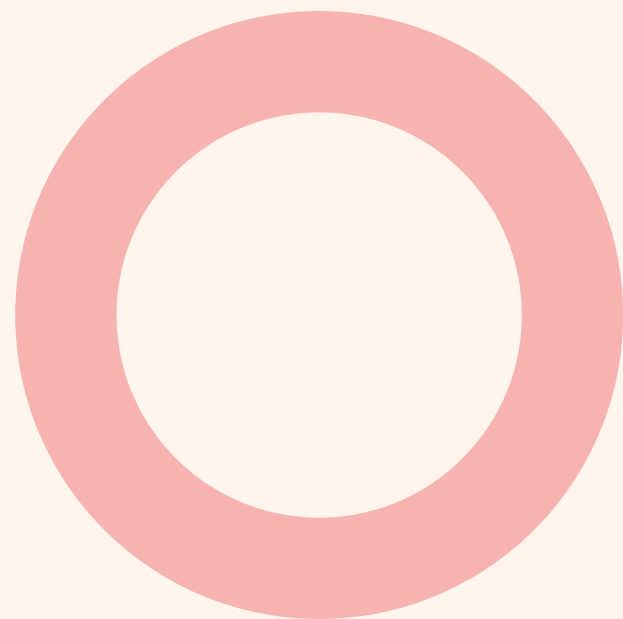
Connetion Pool 크기를 크게 잡는 것이 좋다.



2번 문제

Thread와 Connection 사이 불균형 발생

자원 사용량 폭증 or Connection **놀고 있는 상황** 발생 가능



3번 문제

Connection Pool 사용 시

Connection 수 부족할 걱정은 하지 않아도 됨



3번 문제

Connection Pool **크기 잘못 설정 시**

요청에 비해 Connection 수 적어

대기 시간 증가, 응답 지연, 성능 저하 발생 가능

